DHT 지속성 및 부트스트래핑 분석

개요

rqbit의 DHT 구현은 네트워크 재시작 시에도 빠른 복구를 위해 라우팅 테이블과 피어 정보를 지속적으로 저장하고, 효율적인 부트스트래핑 메커니즘을 통해 DHT 네트워크에 참여합니다. 이 문서는 DHT 상태 저장/복원 메커니즘, 부트스트래핑 과정, 그리고 네트워크 분할 복구 방식을 상세히 분석합니다.

DHT 지속성 메커니즘

PersistentDht 구조체

DHT 지속성을 관리하는 핵심 구조체:

```
pub struct PersistentDht {
// config_filename: PathBuf, // 설정 파일 경로 (주석 처리됨)
}
```

```
#[derive(Default)]
pub struct PersistentDhtConfig {
pub dump_interval: Option<Duration>, // 덤프 간격 (기본값: 60초)
pub config_filename: Option<PathBuf>, // 설정 파일 경로
}
```

직렬화 데이터 구조

저장되는 DHT 상태의 구조:

```
#[derive(Serialize, Deserialize)]
struct DhtSerialize<Table, PeerStore> {
addr: SocketAddr, // 리스닝 주소
table: Table, // IPv4 라우팅 테이블
table_v6: Option<Table>, // IPv6 라우팅 테이블 (하위 호환성)
peer_store: Option<PeerStore>, // 피어 저장소 (하위 호환성)
}
```

기본 저장 위치

DHT 상태는 시스템의 캐시 디렉토리에 저장됩니다:

```
impl PersistentDht {
pub fn default_persistence_filename() -> anyhow::Result<PathBuf> {
let dirs = get_configuration_directory("dht")?;
let path = dirs.cache_dir().join("dht.json"); // ~/.cache/dht/dht.json
Ok(path)
}
}
```

상태 저장 과정

주기적 덤프 메커니즘

DHT 상태를 주기적으로 파일에 저장하는 과정:

```
fn dump_dht(dht: &Dht, filename: &Path, tempfile_name: &Path) -> anyhow::Result<

// 임시 파일에 먼저 쓰기

let file = OpenOptions::new()

.truncate(true)

.create(true)

.write(true)

.open(tempfile_name)?;

let mut file = BufWriter::new(file);



let addr = dht.listen_addr();

// 라우팅 테이블과 피어 저장소를 JSON으로 직렬화

match dht.with_routing_tables(|v4, v6| {

serde_json::to_writer(

&mut file,

&DhtSerialize {

addr,

table: v4,

table_v6: Some(v6),

peer_store: Some(&dht.peer_store),

},
```

```
)
}) {
Ok(_) => trace!("dumped DHT to {:?}", &tempfile_name),
Err(e) => return Err(e).context("error serializing DHT routing table"),
}

// 원자적 파일 교체 (임시 파일 -> 실제 파일)
std::fs::rename(tempfile_name, filename)?;
Ok(())
}
```

백그라운드 저장 태스크

DHT 생성 시 백그라운드에서 주기적으로 상태를 저장하는 태스크가 시작됩니다:

```
spawn_with_cancel::<anyhow::Error>(
debug_span!("dht_persistence"),
"dht_persistence",
dht.cancellation_token().clone(),
{
let dht = dht.clone();
let dump_interval = config.dump_interval
.unwrap_or_else(|| Duration::from_secs(60)); // 기본 60초 간격
async move {
```

```
let tempfile_name = {
let file_name = format!("dht.json.tmp.{}", std::process::id());
let mut tmp = config_filename.clone();
tmp.set_file_name(file_name);
tmp
};

loop {
tokio::time::sleep(dump_interval).await;

match dump_dht(&dht, &config_filename, &tempfile_name) {
Ok(_) => trace!(filename=?config_filename, "dumped DHT"),
Err(e) => error!(filename=?config_filename, "error dumping DHT: {:#}", e)
}
}
}
},
);
```

상태 복원 과정

시작 시 상태 로딩

DHT 시작 시 저장된 상태를 복원하는 과정:

```
let de = match OpenOptions::new().read(true).open(&config_filename) {
Ok(dht_json) => {
let reader = BufReader::new(dht_json);
match serde_json::from_reader::<_, DhtSerialize<RoutingTable, PeerStore>>(reader
Ok(mut r) => {
info!(filename=?config_filename, "loaded DHT routing table from");
// rqbit 9 업그레이드를 위한 패치
if r.addr.ip() == Ipv4Addr::UNSPECIFIED {
warn!("patching DHT listen IP address for rqbit 9 upgrade: 0.0.0.0 -> [::]");
r.addr.set_ip(Ipv6Addr::UNSPECIFIED.into());
}
Some(r)
}
Err(e) => {
warn!(filename=?config_filename, "DHT: cannot deserialize routing table: {:#}",
None
}
}
}
Err(e) => match e.kind() {
std::io::ErrorKind::NotFound => None, // 파일이 없으면 새로 시작
_ => return Err(e).context("error reading config file"),
```

```
},

};
```

복원된 데이터 활용

복원된 데이터를 사용하여 DHT 설정을 구성:

```
let (listen_addr, routing_table, peer_store) = de

.map(|de| (Some(de.addr), Some(de.table), de.peer_store))

.unwrap_or((None, None, None));


let peer_id = routing_table.as_ref().map(|r| r.id());


let dht_config = DhtConfig {

peer_id, // 이전 피어 ID 재사용

routing_table, // IPv4 라우팅 테이블 복원

listen_addr, // 이전 리스닝 주소 재사용

peer_store, // 피어 저장소 복원

cancellation_token,

bind_device,

..Default::default()

};
```

부트스트래핑 메커니즘

기본 부트스트랩 노드

DHT는 잘 알려진 부트스트랩 노드들을 사용합니다:

```
pub static DHT_BOOTSTRAP: &[&str] = &[
"dht.transmissionbt.com:6881",
"dht.libtorrent.org:25401"
];
```

부트스트래핑 과정

부트스트래핑은 다음 단계로 진행됩니다:

1. **호스트명 해석**: DNS를 통해 부트스트랩 노드의 IP 주소 획득
2. **IPv4/IPv6 분리**: 주소 타입별로 별도 처리
3. **find_node 쿼리**: 자신의 ID로 find_node 쿼리 전송
4. **라우팅 테이블 구축**: 응답받은 노드들로 라우팅 테이블 초기화

```
async fn bootstrap_hostname(&self, hostname: &str) -> crate::Result<()> {
// DNS 해석
let addrs = tokio::net::lookup_host(hostname)
.await
.map_err(|err| Error::lookup(hostname, err))?
.collect::<Vec<_>>();
// IPv4와 IPv6 주소로 분리하여 병렬 처리
let v4 = RecursiveRequest::find_node_for_routing_table(
```

```
self.dht.clone(),
self.dht.id, // 자신의 ID로 검색
addrs.iter().copied().filter(|a| a.is_ipv4()),
);


let v6 = RecursiveRequest::find_node_for_routing_table(
self.dht.clone(),
self.dht.id,
addrs.iter().copied().filter(|a| a.is_ipv6()),
);


// 둘 중 하나라도 성공하면 부트스트래핑 성공
let (v4, v6) = tokio::join!(v4, v6);
v4.or(v6)
}
```

지수 백오프를 통한 재시도

부트스트래핑 실패 시 지수 백오프를 사용한 재시도:

```
async fn bootstrap_hostname_with_backoff(&self, addr: &str) -> crate::Result<()>
let backoff = ExponentialBuilder::new()
.with_max_delay(Duration::from_secs(60)) // 최대 60초 지연
.with_jitter() // 지터로 동시 요청 방지
```

```
.with_total_delay(Some(Duration::from_secs(86400))) // 24시간 총 시도
.without_max_times(); // 무제한 재시도

(|| self.bootstrap_hostname(addr))
.retry(backoff)
.notify(|error, retry_in| {
warn!(?retry_in, ?addr, "error in bootstrap: {error:#}");
})
.await
}
```

다중 부트스트랩 노드 처리

여러 부트스트랩 노드를 병렬로 처리:

```
async fn bootstrap(&self, bootstrap_addrs: &[String]) -> crate::Result<()> {
let mut futs = FuturesUnordered::new();

// 모든 부트스트랩 주소에 대해 병렬 시도
for addr in bootstrap_addrs.iter() {
futs.push(self.bootstrap_hostname_with_backoff(addr));
}
let mut successes = 0;
while let Some(resp) = futs.next().await {
```

```
if resp.is_ok() {
successes += 1
}
}
// 하나라도 성공하면 부트스트래핑 성공
if successes == 0 {
return Err(Error::BootstrapFailed);
}
Ok(())
}
```

네트워크 분할 복구

버킷 새로고침을 통한 복구

네트워크 분할이나 장기간 오프라인 후 복구를 위한 메커니즘:

```
async fn bucket_refresher(&self, is_v4: bool) -> crate::Result<()> {
let mut interval = tokio::time::interval(INACTIVITY_TIMEOUT); // 15분 간격
loop {
interval.tick().await;
let now = now();
let mut found = 0;
```

```
for bucket in table.read().iter_buckets() {
// 15분 이상 새로고침되지 않은 버킷 찾기
if now - bucket.leaf.last_refreshed < INACTIVITY_TIMEOUT {
continue;
}
found += 1;
// 버킷 범위 내의 랜덤 ID 생성
let random_id = bucket.random_within();
// 가장 가까운 노드들에게 find_node 쿼리
let addrs = table
.read()
.sorted_by_distance_from(random_id, now())
.iter()
.map(|n| n.addr())
.take(8)
.collect::<Vec<_>>();
// 재귀적 find_node로 버킷 새로고침
RecursiveRequest::find_node_for_routing_table(
self.dht.clone(),
random_id,
addrs.into_iter()
).await?;
}
```

```
trace!("iteration {}, refreshing {} buckets", iteration, found);
}
}
```

노드 생존성 확인

의심스러운 노드들을 주기적으로 확인하여 라우팅 테이블 품질 유지:

```
async fn pinger(&self, is_v4: bool) -> crate::Result<()> {
let mut interval = tokio::time::interval(INACTIVITY_TIMEOUT / 4); // 3.75분 간격
loop {
interval.tick().await;
let now = now();
for node in table.read().iter() {
if matches!(node.status(now), NodeStatus::Questionable | NodeStatus::Unknown) {
// 의심스럽거나 알 수 없는 노드에 ping
table.write().mark_outgoing_request(&node.id(), now);
match self.dht.request(Request::Ping, node.addr()).await {
Ok(_) => {
table.write().mark_response(&node.id(), now);
},
Err(_) => {
table.write().mark_error(&node.id());
}
```

```
}
}
}
}
}
```

안정성 보장 방식

원자적 파일 쓰기

데이터 손실을 방지하기 위해 임시 파일을 사용한 원자적 쓰기:

```
// 1. 임시 파일에 쓰기
let tempfile_name = {
let file_name = format!("dht.json.tmp.{}", std::process::id());
let mut tmp = config_filename.clone();
tmp.set_file_name(file_name);
tmp
};

// 2. 데이터 직렬화
serde_json::to_writer(&mut file, &dht_data)?;

// 3. 원자적 파일 교체
```

```
std::fs::rename(tempfile_name, filename)?;
```

하위 호환성 유지

이전 버전과의 호환성을 위한 처리:

```
// IPv6 라우팅 테이블이 없는 이전 버전 지원
table_v6: Option<Table>,

// 피어 저장소가 없는 이전 버전 지원
peer_store: Option<PeerStore>,

// rqbit 9 업그레이드를 위한 주소 패치
if r.addr.ip() == Ipv4Addr::UNSPECIFIED {
warn!("patching DHT listen IP address for rqbit 9 upgrade: 0.0.0.0 -> [::]");
r.addr.set_ip(Ipv6Addr::UNSPECIFIED.into());
}
```

오류 처리 및 복구

파일 읽기/쓰기 오류에 대한 견고한 처리:

```
match serde_json::from_reader::<_, DhtSerialize<RoutingTable, PeerStore>>(reader
Ok(r) => {
info!("loaded DHT routing table from {:?}", config_filename);
```

```
Some(r)

}

Err(e) => {

warn!("DHT: cannot deserialize routing table: {:#}", e);

None // 오류 시 새로 시작

}

}
```

성능 최적화

비동기 I/O

모든 파일 I/O와 네트워크 작업을 비동기로 처리하여 블로킹 방지:

```
// 비동기 DNS 해석

let addrs = tokio::net::lookup_host(hostname).await?;



// 비동기 파일 쓰기 (백그라운드 태스크에서)

tokio::time::sleep(dump_interval).await;
```

메모리 효율성

- **지연 로딩**: 필요할 때만 상태 복원
- **압축 직렬화**: JSON 대신 더 효율적인 형식 고려 가능
- **선택적 저장**: 중요한 데이터만 선별적 저장

이러한 구현을 통해 rqbit의 DHT는 네트워크 재시작과 장애 상황에서도 빠른 복구와 안정적인 서비스를 제공할 수 있습니다.